

1 Laden von Konstanten in Register

Anw-01 Kleine Konstanten können direkt im Maschinenwort untergebracht werden.

Anw-02 Registerinhalte (im Hex-Format) nach Ausführung von Anw-02:

→ -128 Zweierkomplement

Register R0				Register R1			
06	00	00	12	44	44	44	80
31	16	15	0	31	16	15	0

Anw-03 Größere Konstante erfordern einen zusätzlichen Speicherplatz zum Speichern des Wertes. Dieser kann automatisch vom Assembler erzeugt werden (Gleichheitszeichen). Durch Auswertung des Disassemblers kann die Adresse des Speicherplatzes bestimmt werden:

Adresse der Konstanten:				Inhalt Register R2 nach Anw-03:			
0800	01	4		12	34	56	78
31	16	15	0	31	16	15	0

Zum Zugriff auf den zusätzlichen Speicherplatz setzt der Assembler eine besondere Assembleranweisung ein. Geben Sie an, welche Anweisung die CPU tatsächlich ausführt:

ldr r2, [pc, #56]@ (0x8000236)

2 Zugriff auf Variable

Anw-04 Variable werden im Datenbereich des Prozessors untergebracht. Auf die Variablen kann nicht direkt zugegriffen werden, sondern es muss

Anw-05 zuvor deren Speicheradresse in ein Register geladen werden:

Adresse v. VariableA (Reg. R0 nach Anw-05)

Inhalt Register R1 nach Anw-05:

Anzahl von Anw-05 geladener Bits:

20	00	00	0C	00	00	12	34
31	16	15	0	31	16	15	0

16

Anw-06 Der Assembler überprüft nicht, ob der Lade/Speicherbefehl mit der Definition der Variable übereinstimmt. Anw-06 lädt 32 Bits, obwohl VariableA nur 16 Bit umfasst:

Inhalt Register R2 nach Anw-06:

Anzahl von Anw-06 geladener Bits:

42	11	12	34
31	16	15	0

32

Anw-07 Die Adresse einer Variable kann auch als Differenz zu einer Basis angegeben werden:

Adresse v. VariableC:

Inhalt von VariableC nach Anw-07:

Anzahl von Anw-07 gespeicherter Bits:

20	00	00	10	34	12	11	47
31	16	15	0	31	16	15	0

32

LE

Dissassembly view: wohin kommen die ganzen $0x08000xxx$
wieso startet er in Zeile 57

#Variable-Variable = #4

Kann ich bei GG verwenden

Nullterminator beendet den String

3 Zugriff auf Felder

Die Elemente eines Feldes sind alle vom gleichen Typ und werden der Reihe nach im Speicher angelegt.

MeinHalbwortFeld:

20	00	00	14
31	16	15	0

Anfangsadresse:

Anzahl Elemente:

Anzahl Bytes pro Element:

DCW

MeinWortFeld:

20	00	00	20
31	16	15	0

Anfangsadresse:

Anzahl Elemente:

Anzahl Bytes pro Element:

DCD

MeinTextFeld:

20	00	00	38
31	16	15	0

Anfangsadresse:

Anzahl Elemente:

Anzahl Bytes pro Element:

DCB
Nulldeterminator

Anw-08 Der Zugriff auf ein bestimmtes Element kann durch das Laden der Anfangsadresse in ein Register und die Angabe der Adressdifferenz für
Anw-09 das jeweilige Element erfolgen.

Anw-10

00	00	00	20
31	16	15	0

Register R1 nach Anw-09:

00	00	60	3C
31	16	15	0

Register R2 nach Anw-10:

Anw-11 Die Adressdifferenz kann auch über ein Register angegeben werden. Sie kann somit während des Programmlaufs verändert werden: *10 in R3*
Anw-12

00	00	00	45
31	16	15	0

Register R4 nach Anw-12:

Nr. (beginnend mit 0) des
von Anw-12 gelesenen Elements:

5

 $0 \times 22 = 0$

Anw-13 Die Basisadresse kann automatisch nach Zugriff auf ein Element erhöht werden.

20	00	00	16
31	16	15	0

Register R0 nach Anw-13

00	00	00	3E
31	16	15	0

Register R5 nach Anw-13:

Nr. (beginnend mit 0) des
von Anw-13 gelesenen Elements:

1

Anw-14

20	00	00	15
31	16	15	0

Register R0 nach Anw-14

00	00	FF	CC
31	16	15	0

Register R6 nach Anw-14:

Nr. (beginnend mit 0) des
von Anw-14 gelesenen Elements:

2

Anw-15

20	00	00	1A
31	16	15	0

Register R0 nach Anw-15

20	00	00	1A
31	16	15	0

Adresse des von Anw-15 überschriebenen Elements:
--

Nr. (beginnend mit 0) des
von Anw-15 überschriebenen Elements:

3

 $\frac{6}{2} = 3$
 $0 \times 1A - 0 \times 14 = 2$

4 Addition und Subtraktion von vorzeichenlosen und vorzeichenbehafteten Integer-Werten

Arithmetik-Befehle haben 3 Operanden. Tragen Sie in das nachfolgende Schema die beteiligten Werte in binärer Form ein. Wandeln Sie die binäre Darstellungen um in vorzeichenbehaftete und vorzeichenlose Dezimalzahlen. Überprüfen Sie die Rechnung. Stimmen die Ergebnisse mit den Erwartungen überein?

Anw-16
Anw-17
Anw-18
Anw-19

		dezimale Interpretation der Bitmuster	
		signed	unsigned
Reg. R1:	00010010000410100001010110011111000	04305413896	00305413896
Reg. R2:	10011110110110100101100110000110	-164773306	02647193990
Reg. R3:	10101111111111110101011111111110	-1342353420	02952673886
31 α $\dagger$ $\dagger$ e 16 15 α $\dagger$ e 0 Kreuzen Sie die gesetzten Bits des Statusregisters an: V C ☒ Z		Ergebnis: ☒ Ok ☐ Nicht Ok	Ergebnis: ☒ Ok ☐ Nicht Ok

Anw-20
Anw-21
Anw-22

		dezimale Interpretation der Bitmuster	
		signed	unsigned
Reg. R4:	11001100000000000000000000000000	0-872415232	03422582064
Reg. R5:	01001110000000000000000000000000	+1308622848	01308622848
Reg. R6:	01111111000000000000000000000000	+2113929616	02113929216
31 16 15 0 Kreuzen Sie die gesetzten Bits des Statusregisters an: ☒ ☒ N Z		Ergebnis: ☐ Ok ☒ Nicht Ok	Ergebnis: ☒ Ok ☐ Nicht Ok

Anw-23
Anw-24
Anw-25

		dezimale Interpretation der Bitmuster	
		signed	unsigned
Reg. R7:	10100111000000000000000000000000	05654311424	00654311424
Reg. R8:	01000101000000000000000000000000	+1157627504	01157627504
Reg. R9:	11100010000000000000000000000000	0-503316480	03721050816
31 16 15 0 Kreuzen Sie die gesetzten Bits des Statusregisters an: V C ☒ Z		Ergebnis: ☒ Ok ☐ Nicht Ok	Ergebnis: ☐ Ok ☒ Nicht Ok